A Comprehensive Guide to LLM Agent Debugging

Gemini Agent

An Exhaustive Walkthrough of AgentDebug (Zhu et al., 2025)

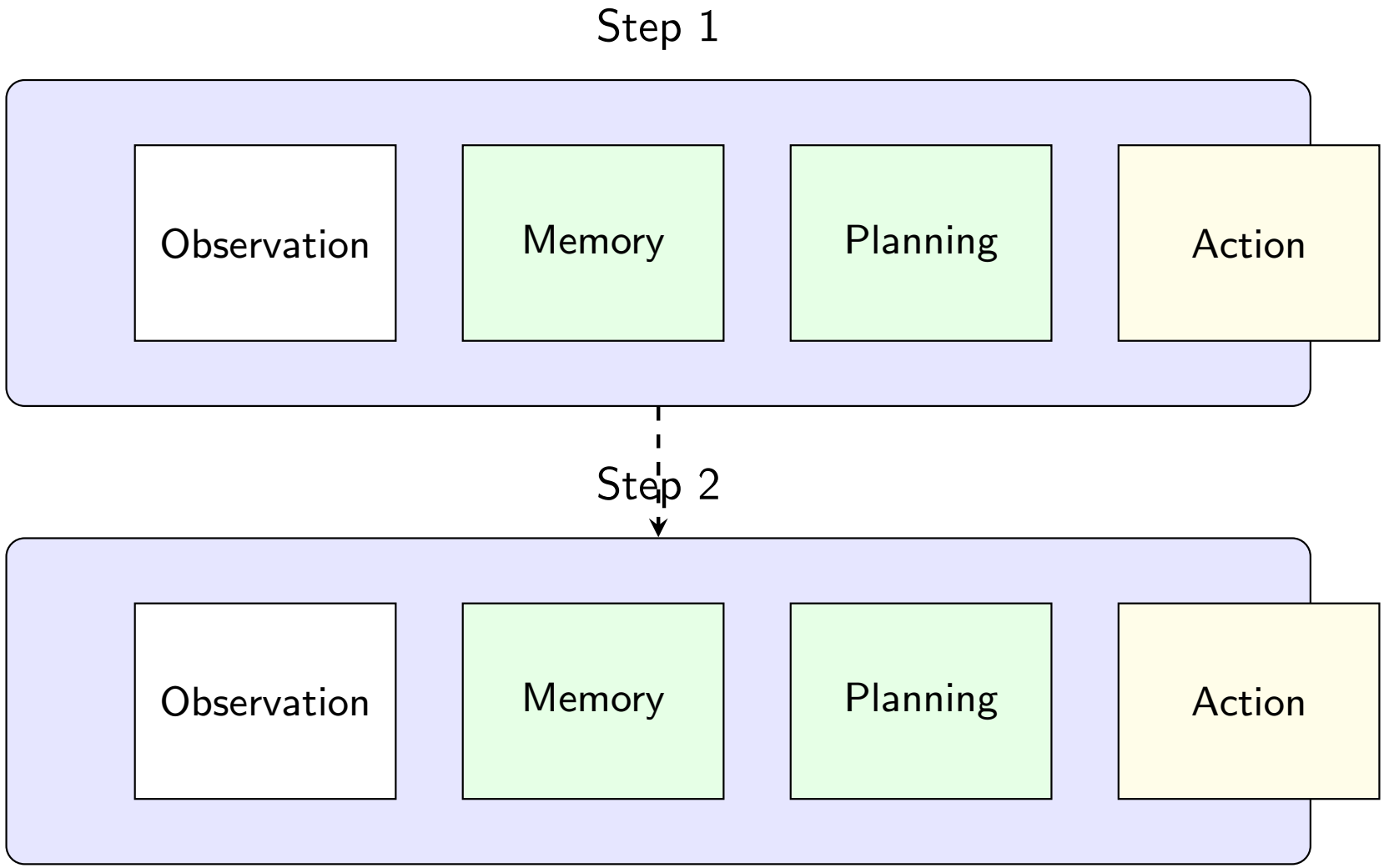
November 13, 2025

Presentation Agenda

1. Understanding the Agent's World: Trajectories & Failures
2. A Deep Dive into the `AgentErrorTaxonomy`
   - Methodology
   - Complete breakdown of all 5 modules and their error types
3. The `AgentDebug` Framework: A Step-by-Step Guide
4. Experimental Validation: Does it Work?
5. Illustrative Cases from Multi-Agent Systems (`MAST`)
6. Conclusion & Key Takeaways

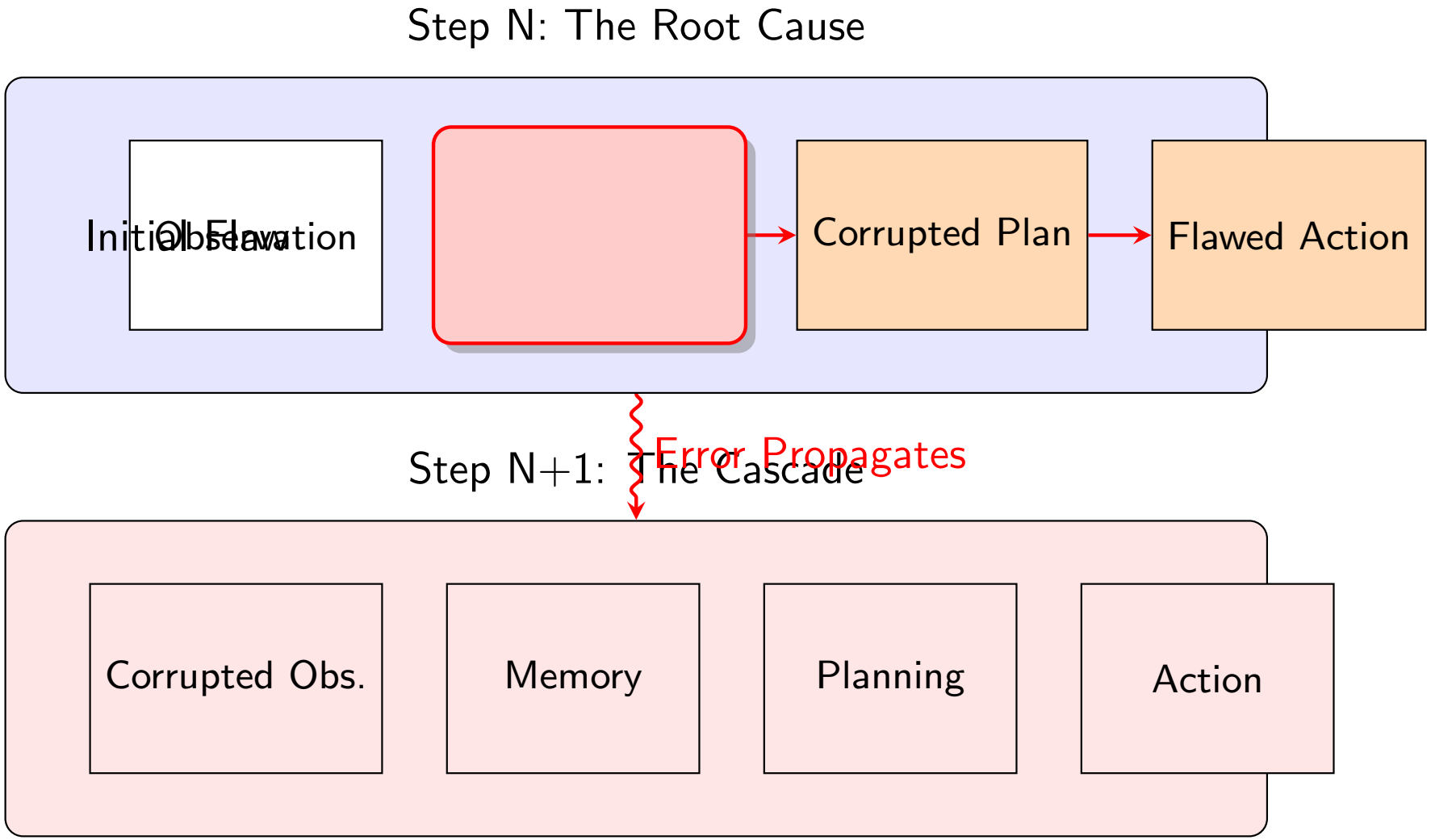
Visualizing an Agent's Trajectory

A **trajectory** is the complete log of an agent's attempt to solve a task. It's a sequence of steps, where each step contains the agent's entire reasoning process.



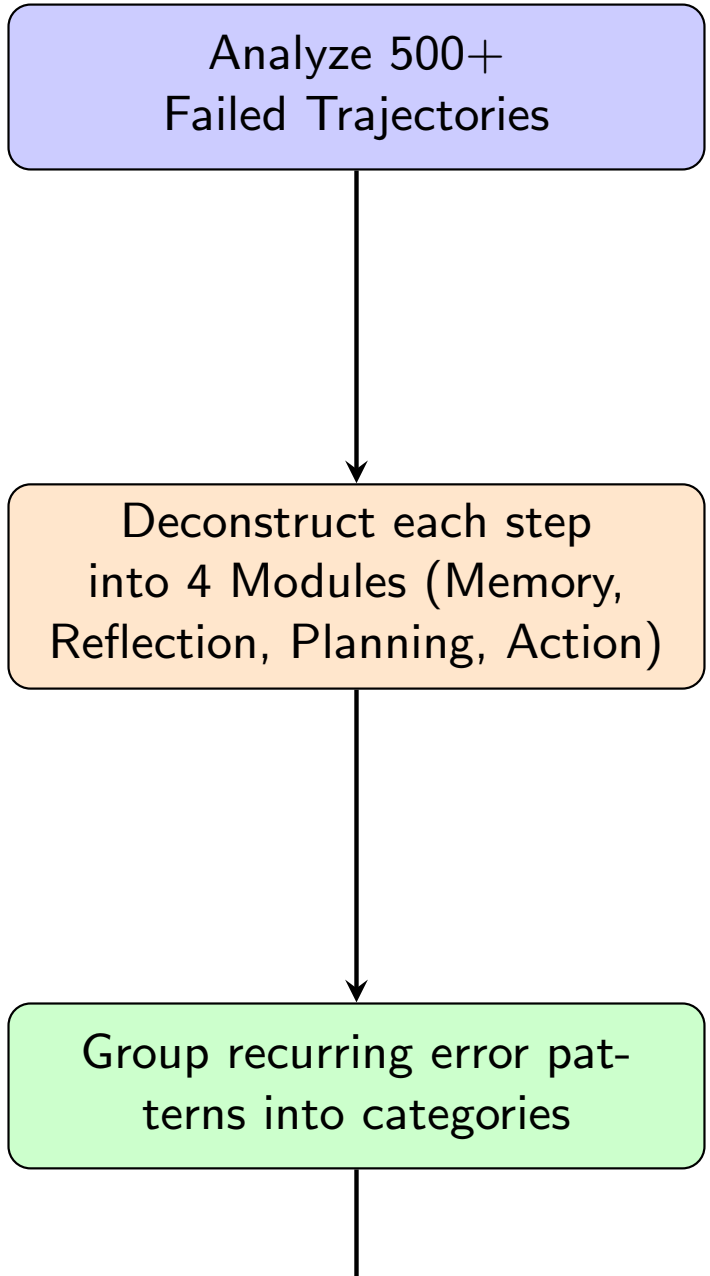
How and Where Failures Appear in a Trajectory

A single flaw in one module can corrupt the entire rest of the step, and cascade into future steps.



Building a Language for Failure: Methodology

To debug failures, we first need a precise vocabulary to describe them.



The Full `AgentErrorTaxonomy` (1/3): Memory

Memory: Errors in recalling or retrieving information.

Over-simplification / Incomplete Summary Summarizes past info too crudely, ignoring critical details.

- *Example:* Agent is told "find a red shirt, size medium." Its memory becomes "find a shirt," forgetting the constraints.

Hallucination (False Memory) Recalls events or states that never happened.

- *Example:* After a failed API call, the agent "remembers" that the call was successful and returned data.

Retrieval Failure Relevant information exists in the agent's history, but it fails to retrieve it when needed.

- *Example:* The user provided an API key on step 1, but on step 5 the agent asks for the key again.

The Full `AgentErrorTaxonomy` (2/3): Reflection & Planning

Reflection: Errors in monitoring progress or interpreting outcomes.

Progress Misassessment Incorrectly evaluates its own progress (e.g., thinks it's done when it's not).

Outcome Misinterpretation Correctly executes an action but misreads the feedback from the environment.

Causal Misattribution Correctly notes a failure but blames the wrong cause.

Planning: Errors in formulating a strategy.

Constraint Ignorance Ignores limits (time, budget, user rules) when forming a plan.

Impossible Action Plans a step that is physically or logically impossible.

Inefficient Planning The plan is overly long, illogical, or wastes steps.

The Full `AgentErrorTaxonomy` (3/3): Action & System

Action: Errors in executing an operation.

Planning-Action Disconnect The chosen action does not align with the agent's stated plan.

Format Error Produces a syntactically invalid action (e.g., malformed JSON).

Parameter Error Generates an action with unreasonable or malformed parameters.

System-level: Failures caused by the external environment or tools.

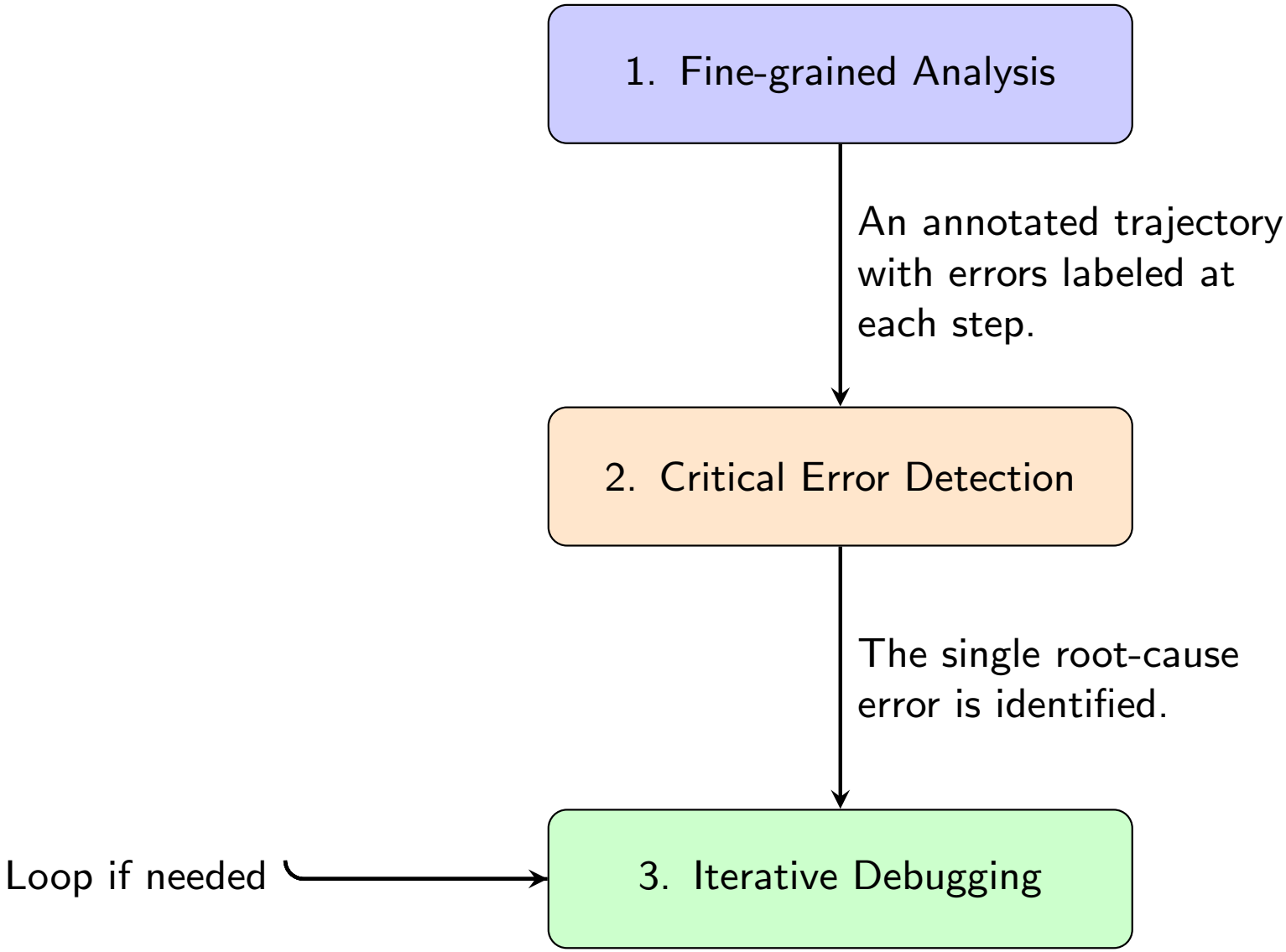
Step Limit Exhaustion Fails by reaching the maximum step count.

Tool Execution Error An external tool/API crashes or returns an error.

Environment Error The environment itself breaks or does not follow expected rules.

The AgentDebug Framework: A Step-by-Step Guide

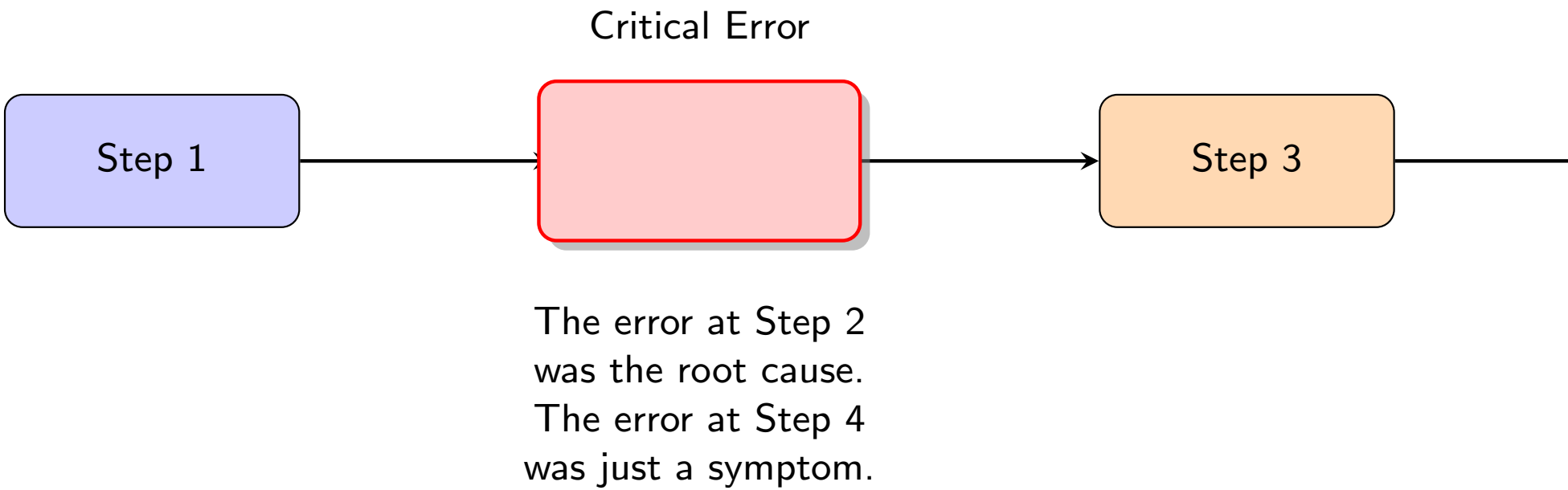
With a taxonomy in place, we can build a system to automatically find and fix errors.



Framework Deep Dive: Critical Error Detection

This is the core of `AgentDebug`. It's not about finding all errors, but about finding the **one** that mattered most.

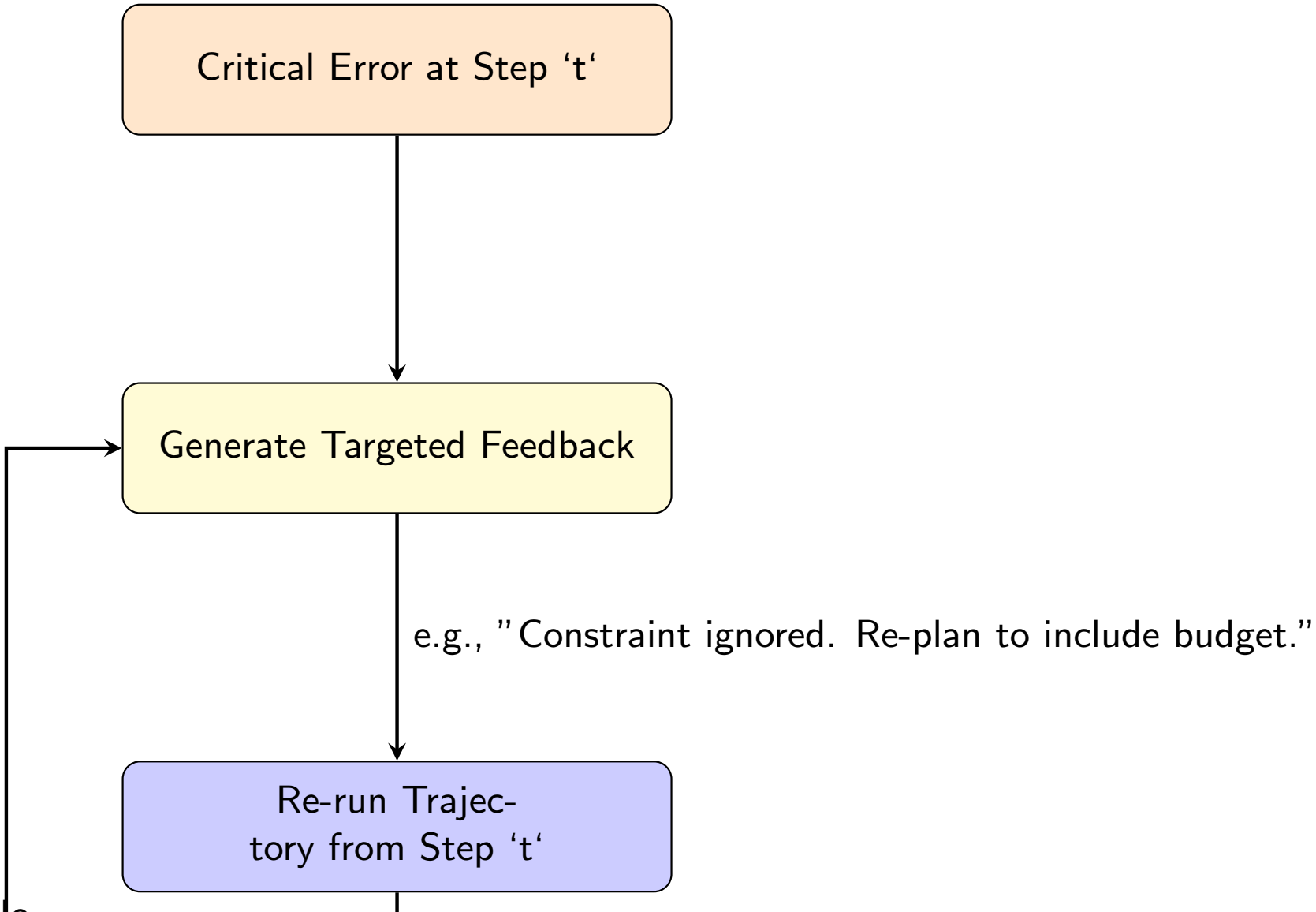
The "Counterfactual" Question: The system asks for each step 't' in the trajectory: *"If the agent had done the right thing at step 't', would the rest of the trajectory have succeeded?"*



The **Critical Error** is the *earliest* step where the answer is "Yes." This prevents wasting time fixing downstream symptoms of an earlier, deeper mistake.

Framework Deep Dive: Iterative Debugging

Once the root cause is found, `AgentDebug` generates targeted feedback and re-runs the agent from that critical point.



Experimental Validation: Does It Work?

Yes. The paper shows that this principled debugging approach significantly improves task success compared to baselines.

Key Finding from Paper's Experiments

— Insert Figure 5 from AgentDebug (Zhu et al., 2025) here. —

placeholder.png

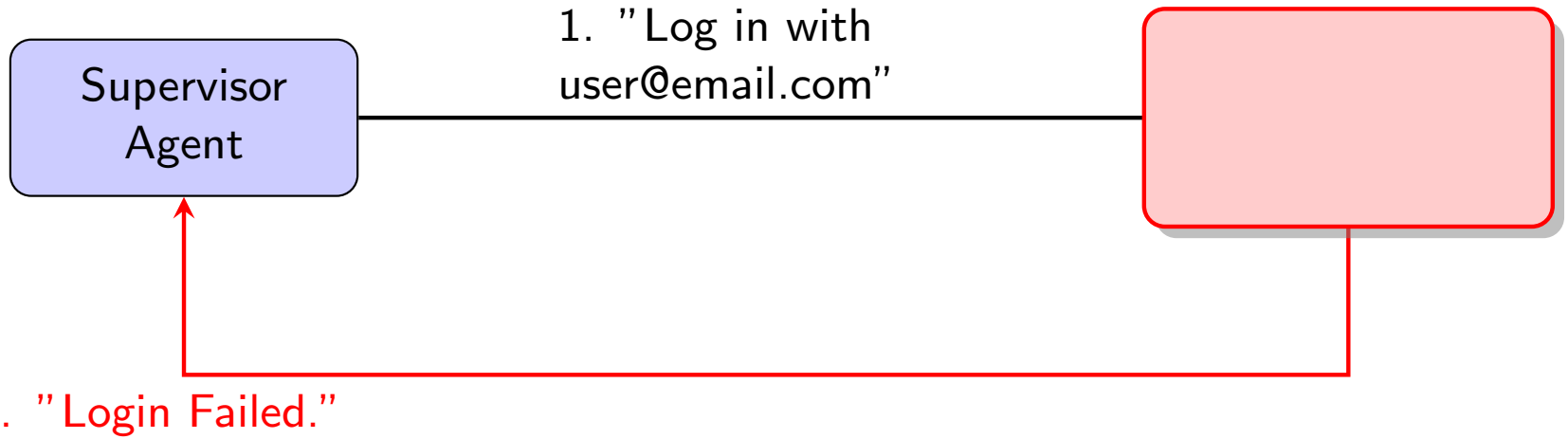
A Glimpse into Multi-Agent Failures

While `AgentDebug` focuses on single agents, the `MAST` paper (Cemri et al., 2025) provides great examples of failures unique to **multi-agent systems**.

In MAS, failures often stem from flawed **communication** and **coordination**, not just individual reasoning.

MAS Failure Case: Information Withholding

Category: Inter-Agent Misalignment (Agents fail to coordinate effectively).



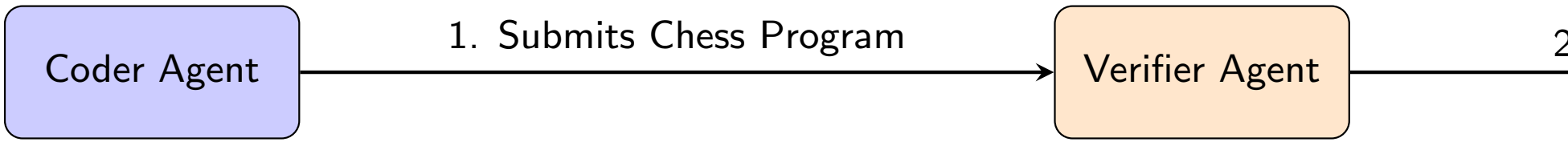
The Flaw

The Login Agent knows the API requires a *phone number*, not an email, for the username. It withholds this critical information, causing repeated failures.

This is a communication failure. The system as a whole fails because the agents do not share necessary context to help each other succeed.

MAS Failure Case: Incorrect Verification

Category: Task Verification (The system fails to properly check its own work).



The Verifier only checks if the code *compiles*. It never runs a test to see if the chess rules are correct.

The system prematurely declares success because its verification process was too superficial.

Conclusion & Key Takeaways

1. **Failure is Systemic, Not Just Local**
   Errors are rarely isolated. They propagate within a single agent's reasoning and emerge from flawed interactions between multiple agents.

2. **A Taxonomy is a Prerequisite for Debugging**
   We need a precise language to describe failures before we can build tools to fix them. The `AgentErrorTaxonomy` provides this for single agents.

3. **Root Cause Analysis is Essential**
   Fixing surface-level symptoms is not enough. Effective debugging, as demonstrated in `AgentDebug`, must find and correct the single, earliest point of failure.

4. **Automation is the Goal**
   Frameworks like `AgentDebug` are the first step towards creating agents that can learn from their own mistakes and become truly robust and reliable.

References I


Zhu, K., Liu, Z., Li, B., et al. (2025).
WHERE LLM AGENTS FAIL AND HOW THEY CAN LEARN FROM FAILURES.
arXiv:2509.25370v1.


Cemri, M., Pan, M. Z., Yang, S., et al. (2025).
Why Do Multi-Agent LLM Systems Fail?.
arXiv:2503.13657v3.

Thank You

Questions?